1.2 核心架构一览

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~140k tokens，输出 ~2.9k tokens（本节）

上一节我们聊的是“OpenClaw 想成为什么”。这一节要回答的是另一个问题：它大致是怎么运转起来的。

先提前说结论。OpenClaw 的整体链路并不神秘，你可以先把它记成一行：消息从不同入口进来，先到 Gateway，再交给 Agent Runtime 处理；如果需要做事，就调用工具；最后把结果再发回去。

写成更直观一点的形式，就是：

```
消息通道 -> Gateway -> Agent Runtime -> Tools -> 回复
```

这一行已经抓住了主干。后面你看到的很多模块，基本都可以挂在这条线上理解。

1.2.1 先从一条消息开始

假设你在 Telegram 里给 OpenClaw 发了一句：“帮我总结一下今天的学习安排。”

这句话进入系统后，不是直接飞到模型那里去。中间会经过几个角色。

第一步，消息通道收到它。这里的“消息通道”可以是 Telegram、WhatsApp、Slack、Discord，也可以是网页聊天入口。你可以把消息通道理解成不同的门。门的样子不一样，进来的人也不一样，但进门之后，总得有人接待。

第二步，Gateway 接住它。Gateway 可以翻成“网关”，你先别被这个词吓到。网关在这里更像一个前台调度员：谁来了、从哪来、要找谁、该走哪条流程，先由它分配。

第三步，Gateway 把需要 AI 处理的任务交给 Agent Runtime。Runtime 就是“运行时”，意思是系统真正执行任务的那一层。它可以理解成助手的大脑工作区：读上下文、组织提示、调用模型、判断要不要用工具。

第四步，如果只是普通问答，Agent Runtime 直接生成回复就行；如果事情需要动手做，比如查天气、读文件、打开浏览器、安排任务，它就会调用对应工具。

第五步，工具把结果交回来，Agent Runtime 组织成更像人话的回应，再通过原来的消息通道发回去。

这就是 OpenClaw 最核心的一次往返。

1.2.2 用一个生活类比看整套架构

如果你对“网关、运行时、工具”这些词还没有画面，我们换个更生活化的比喻。

把 OpenClaw 想成一家小而灵活的工作室。

- 消息通道像前门、侧门、快递口，不同的人从不同入口找上门。
- Gateway 像前台调度员，负责登记、分流、安排。
- Agent Runtime 像真正干活的核心助手，负责理解需求、思考步骤、决定怎么处理。
- Tools 像助手手边的一整排工具箱，里面有浏览器、终端、文件处理、定时任务、设备能力等等。
- 最后的回复，就是助手把结果整理好，再通过来时的那扇门递回去。

这个比喻的好处是，你能立刻看出分工。

前台不负责亲自干活，它主要负责接待和调度。核心助手不关心人是从正门还是侧门进来的，它关心的是“这件事该怎么做”。工具箱也不负责理解用户，它只在被调用时完成具体动作。

这种分工很关键。因为一旦系统变复杂，你就不希望每个部分都什么都管。那样最后一定会缠成一团。

1.2.3 文字版架构图

我们把刚才那条链路画成一个更完整的文字图：

```
┌──────────────────────────────────────────┐
│                 消息通道层                 │
│  Telegram / WhatsApp / Slack / Discord /   │
│  WebChat / 其他入口                        │
└──────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────┐
│                 Gateway 网关               │
│  - 接收消息                                │
│  - 识别来源                                │
│  - 路由到正确会话                          │
│  - 协调后续流程                            │
└──────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────┐
│            Agent Runtime 运行时            │
│  - 读取上下文                              │
│  - 调用模型                                │
│  - 判断是否需要工具                        │
│  - 组织回复                                │
└──────────────────────────────────────────┘
                      │
          ┌───────────┴───────────┐
          ▼                       ▼
┌──────────────────┐   ┌──────────────────────┐
│   直接文本回复    │   │      Tools 工具层     │
│   不需要外部动作  │   │  文件 / 终端 / 浏览器 │
└──────────────────┘   │  定时任务 / 设备能力  │
          │            └──────────────────────┘
          └───────────┬───────────┘
                      ▼
┌──────────────────────────────────────────┐
│                 回复输出层                 │
│     沿原通道发回用户，并保留会话连续性      │
└──────────────────────────────────────────┘
```

你现在不需要记图里每个词，只需要看出一个思路：OpenClaw 把“接消息”“想事情”“做动作”“回结果”拆成了不同层次。这就是它比较清爽的地方。

1.2.4 Gateway 到底管什么

Gateway 是这一套里最容易被低估的部分。很多新手会下意识觉得：AI 系统里最重要的肯定是模型本身。模型当然重要，但在 OpenClaw 这种系统里，真正把事情串起来的，经常是 Gateway。

它至少管四件事。

第一，接入口。不同消息平台接入方式各不相同，但 Gateway 负责把这些差异压平，让后面的系统不用每次都重新适配。

第二，做路由。路由可以简单理解成“该把消息送到哪里”。比如同样是一句消息，它可能属于某个已有会话，也可能要开一个新会话；它可能该交给主助手，也可能该交给某个专门助手。

第三，管会话。会话就是一次持续对话的上下文容器。你可以把它想成一条不断延长的聊天线。Gateway 需要知道：这句话接在哪段历史后面，什么时候该重置，什么时候该继续。

第四，做协调。AI 回复不是系统里唯一发生的事。还有工具调用、状态更新、外部客户端连接、定时任务触发，这些都需要一个中枢来调度。Gateway 就承担这个中枢角色。

如果前面那个工作室比喻你觉得顺手，那 Gateway 就不只是前台，它更像“前台加调度台再加总机房”的混合体。

1.2.5 Agent Runtime 为什么是“大脑”

如果说 Gateway 负责把事情接进来、安排好，那真正“想”和“做决策”的地方，就是 Agent Runtime。

它做的事，远不只是把用户那句话原样发给模型。中间通常会有一整套准备动作：

- 读取当前会话的历史内容。
- 结合系统设定，告诉模型这个助手应该怎么说话、怎么做事。
- 判断这次任务需不需要查资料、开工具、调用外部能力。
- 在工具返回结果后，再决定下一步要不要继续调用，还是直接回答。

你可以把 Agent Runtime 理解成“模型外面的那层智能组织”。模型本身很强，但如果没有这层运行时来喂上下文、接工具、管流程，模型就更像一个非常聪明但坐在空房间里的人。Agent Runtime 做的，就是把这间空房间变成一个能工作的办公室。

这里顺便解释一个词：上下文。上下文不是玄学，就是“模型在当前这一轮能看到的背景信息”。它可能包括之前的聊天记录、系统规则、工具结果、记忆内容等等。为什么上下文重要？因为没有它，助手每次都像失忆重来。

1.2.6 Tools 是什么，为什么它们这么关键

很多人刚开始会把工具理解成“额外插件”。这不算错，但在 OpenClaw 里，工具的地位其实更高一些。因为没有工具，助手大多数时候只能停留在“会说”；有了工具，它才开始“会做”。

这里的 Tools，可以简单分成几类：

- 文件和终端类：读写内容、执行命令、整理工作区。
- 浏览器类：打开网页、读取页面、做交互。
- 自动化类：定时任务、批量处理、计划执行。
- 设备和环境类：连接不同节点，使用摄像头、屏幕等能力。

你不用一开始就把每个工具都记住，只要先明白：工具系统是 OpenClaw 从“聊天助手”变成“行动助手”的关键一跳。

这里还有一个容易忽略的点。工具不是用户直接操作的主界面，它更像助手自己的手脚。你提需求，Agent Runtime 决定要不要动用哪只手、哪件工具，然后再把结果整理给你。这个层次感很重要。

1.2.7 一条消息的完整流动过程

现在我们把所有角色串回一次完整流程，用最朴素的话走一遍。

1. 你在某个消息入口发来一句话。
2. 对应通道把消息带进 OpenClaw。
3. Gateway 判断这是谁发来的、该落到哪个会话里。
4. Gateway 把任务交给 Agent Runtime。
5. Agent Runtime 读取历史和设定，开始组织这次思考。
6. 如果只需要文字回答，就直接生成回复。
7. 如果需要外部动作，就去调用一个或多个工具。
8. 工具把执行结果返回给 Agent Runtime。
9. Agent Runtime 再把这些结果整理成适合用户阅读的回应。
10. Gateway 通过原来的消息通道把回应发回去。

你会发现，这条路虽然不短，但每一步都挺自然。OpenClaw 的架构不是靠炫技复杂，而是因为“个人 AI 助手”这件事本来就比“单轮问答”多几层现实需求。

1.2.8 初学者最该记住的三个点

到这里，如果你已经有点信息量过载，没关系。先记住三个最值钱的点。

第一，Gateway 是系统中枢。它负责把不同入口接进来，把消息送到正确的地方，再把结果送回去。

第二，Agent Runtime 是助手的大脑工作区。它不只是调用模型，还负责组织上下文、判断是否需要工具、协调多步处理。

第三，Tools 让 OpenClaw 从“能聊”升级到“能做”。这也是它和普通聊天界面最不像的地方之一。

等你后面继续读，会发现很多看起来零散的模块，其实都能挂回这三句话上。只要主干清楚，后面的细节就不会乱。

本节小结

1. OpenClaw 的核心链路可以先记成“消息通道 -> Gateway -> Agent Runtime -> Tools -> 回复”。
2. Gateway 像前台调度员，负责接入口、做路由、管会话、协调系统里的各类动作。
3. Agent Runtime 是助手真正运转的大脑区，负责读上下文、调模型、决定是否调用工具。
4. Tools 是助手的手脚，没有它们，AI 大多只能停留在回答问题；有了它们，才开始具备行动能力。
5. 这一整套架构的核心目标，不是把系统做复杂，而是让个人 AI 助手真正能长期工作、持续协作。